Coding Test - Solutions

Name: Avnish Chauhan

Q1. Product of Array Except Self

Approach:

Calculate prefix products and suffix products for each index without using division, then multiply them together.

Code:

```
def productExceptSelf(nums):
    n = len(nums)
    answer = [1] * n
    prefix = 1
    for i in range(n):
        answer[i] = prefix
        prefix *= nums[i]
    suffix = 1
    for i in range(n - 1, -1, -1):
        answer[i] *= suffix
        suffix *= nums[i]
    return answer

print(productExceptSelf([1, 2, 3, 4]))  # [24, 12, 8, 6]
```

Time Complexity: O(n) Space Complexity: O(1) extra

Q2. Longest Consecutive Sequence

Approach:

Add all numbers to a set. For each number that is the start of a sequence (num-1 not in set), count how long the consecutive run is.

Code:

```
def longestConsecutive(nums):
    num_set = set(nums)
    longest = 0
    for num in num_set:
        if num - 1 not in num_set:
            length = 1
            current = num
            while current + 1 in num_set:
                current += 1
                length += 1
            longest = max(longest, length)
    return longest

print(longestConsecutive([100, 4, 200, 1, 3, 2]))  # 4
```

Time Complexity: O(n) Space Complexity: O(n)

Q3. 3Sum

Approach:

Sort the array. Fix one number and use two pointers to find pairs that sum to its negative, skipping duplicates.

Code:

```
def threeSum(nums):
```

```
    nums.sort()
    result = []
    n = len(nums)
    for i in range(n - 2):
        if i > 0 and nums[i] == nums[i - 1]:
            continue
        if nums[i] > 0:
            break
        left, right = i + 1, n - 1
        while left < right:
            total = nums[i] + nums[left] + nums[right]
            if total < 0:
                left += 1
            elif total > 0:
                right -= 1
            else:
                result.append([nums[i], nums[left], nums[right]])
                left += 1
                right -= 1
                while left < right and nums[left] == nums[left - 1]:
                    left += 1
                while left < right and nums[right] == nums[right + 1]:
                    right -= 1
    return result

print(threeSum([-1, 0, 1, 2, -1, -4]))  # [[-1,-1,2],[-1,0,1]]
```

Time Complexity: O(n^2) Space Complexity: O(1) extra

Q4. Daily Temperatures

Approach:

Use a stack of indices with decreasing temperatures. When a warmer day is found, pop and record the day difference.

Code:

```
def dailyTemperatures(temperatures):
    n = len(temperatures)
    answer = [0] * n
    stack = []
    for i, temp in enumerate(temperatures):
        while stack and temperatures[stack[-1]] < temp:
            prev = stack.pop()
            answer[prev] = i - prev
        stack.append(i)
    return answer

print(dailyTemperatures([73,74,75,71,69,72,76,73]))
# [1, 1, 4, 2, 1, 1, 0, 0]
```

Time Complexity: O(n) Space Complexity: O(n)

Q5. Kth Largest Element in an Array

Approach:

Keep a min-heap of size k. After adding all numbers, the smallest item in the heap is the kth largest overall.

Code:

```
import heapq
```

```
def findKthLargest(nums, k):
    heap = []
    for num in nums:
        heapq.heappush(heap, num)
        if len(heap) > k:
            heapq.heappop(heap)
    return heap[0]

print(findKthLargest([3,2,1,5,6,4], 2))  # 5
```

Time Complexity: O(n log k) Space Complexity: O(k)

Q6. Merge K Sorted Linked Lists

Approach:

Push the head of each list into a min-heap. Repeatedly pop the smallest node, add it to the result, and push its next node.

Code:

```
import heapq

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def mergeKLists(lists):
    heap = []
    for i, node in enumerate(lists):
        if node:
            heapq.heappush(heap, (node.val, i, node))
    dummy = ListNode()
    tail = dummy
    while heap:
        val, i, node = heapq.heappop(heap)
        tail.next = node
        tail = tail.next
        if node.next:
            heapq.heappush(heap, (node.next.val, i, node.next))
    return dummy.next

# [[1,4,5],[1,3,4],[2,6]] -> [1,1,2,3,4,4,5,6]
```

Time Complexity: O(N log k) Space Complexity: O(k)

Q7. Course Schedule II

Approach:

Build a graph and count in-degrees. Use Kahn's algorithm (BFS) starting with courses that have no prerequisites. If not all courses are processed, return an empty list (a cycle exists).

Code:

```
from collections import deque, defaultdict

def findOrder(numCourses, prerequisites):
    graph = defaultdict(list)
    indegree = [0] * numCourses
```

```
    for course, prereq in prerequisites:
        graph[prereq].append(course)
        indegree[course] += 1
    queue = deque(c for c in range(numCourses) if indegree[c] == 0)
    order = []
    while queue:
        course = queue.popleft()
        order.append(course)
        for nxt in graph[course]:
            indegree[nxt] -= 1
            if indegree[nxt] == 0:
                queue.append(nxt)
    return order if len(order) == numCourses else []

print(findOrder(4, [[1,0],[2,0],[3,1],[3,2]]))
# e.g. [0, 1, 2, 3]
```

Time Complexity: O(V + E) Space Complexity: O(V + E)

Q8. Lowest Common Ancestor of a Binary Tree

Approach:

Recurse down the tree. If a node is null or equals p or q, return it. If both left and right recursion return non-null, the current node is the LCA.

Code:

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def lowestCommonAncestor(root, p, q):
    if root is None or root == p or root == q:
        return root
    left = lowestCommonAncestor(root.left, p, q)
    right = lowestCommonAncestor(root.right, p, q)
    if left and right:
        return root
    return left if left else right

# p = 5, q = 1 -> LCA = 3
```

Time Complexity: O(n) Space Complexity: O(h)

Q9. Word Search

Approach:

Try starting from every matching cell and use DFS/backtracking to move up, down, left, right, marking visited cells and restoring them if the path fails.

Code:

```
def exist(board, word):
    rows, cols = len(board), len(board[0])
    def dfs(r, c, i):
        if i == len(word):
            return True
        if (r < 0 or r >= rows or c < 0 or c >= cols
                or board[r][c] != word[i]):
```

```
            return False
        temp = board[r][c]
        board[r][c] = '#'
        found = (dfs(r+1, c, i+1) or dfs(r-1, c, i+1) or
                 dfs(r, c+1, i+1) or dfs(r, c-1, i+1))
        board[r][c] = temp
        return found
    for r in range(rows):
        for c in range(cols):
            if dfs(r, c, 0):
                return True
    return False

board = [['A','B','C','E'],['S','F','C','S'],['A','D','E','E']]
print(exist(board, 'ABCCED'))  # True
```

Time Complexity: O(m*n*4^L) Space Complexity: O(L)

Q10. Longest Increasing Subsequence

Approach:

Keep a list 'tails' of smallest possible ending values for increasing subsequences of each length. Use binary search to find where each number fits, replacing or extending the list.

Code:

```
import bisect

def lengthOfLIS(nums):
    tails = []
    for num in nums:
        pos = bisect.bisect_left(tails, num)
        if pos == len(tails):
            tails.append(num)
        else:
            tails[pos] = num
    return len(tails)

print(lengthOfLIS([10,9,2,5,3,7,101,18]))  # 4
```

Time Complexity: O(n log n) Space Complexity: O(n)